Grover's Search Algorithm

From looking through a list to quantum amplitude amplification — explained step by step

CUDA-Q Quantum Algorithm Benchmarking Suite

For first-year engineering students

Quantum results come from NVIDIA CUDA-Q on its CPU simulators, checked against theory.
No GPU or quantum hardware run is shown.

What you will learn today

1. What a **search problem** is and what it costs classically
2. The math toolkit: **complex numbers**, vectors, reflections
3. **Qubits** in ten minutes
4. **Grover's algorithm**: oracle, diffusion, iteration
5. The **geometric picture** and every derivation
6. Several solutions, unknown M, amplification, counting
7. How to **build the circuit** from simple gates
8. The **CUDA-Q notebook**, measured results, noise and limits

You only need

- school algebra (powers, sums)
- vectors and matrices (multiplying a matrix by a vector)
- a little trigonometry (sin, cos)

Everything else is explained on the way.

Symbols used in every slide

symbol	meaning
n	number of qubits
$N = 2^n$	number of items searched
M	number of marked items
x	an item, as an n-bit string
x^*	the marked item (when $M = 1$)
$f(x)$	1 if x is marked, otherwise 0
U_ω	phase oracle
D	diffusion operator
$G = DU_\omega$	one Grover iteration
k	number of iterations
θ	angle with $\sin\theta = \sqrt{M/N}$

Reading convention

Qubit 1 (top wire) is the **most significant bit**: $|101\rangle$ is the number 5.

Notebook naming

`Grover.ipynb` uses N for the number of qubits, $M = 2^N$ for the items, and O for the oracle. The formulas are the same; only the letters differ.

What is a search problem?

- We have N possible candidates (the **search space**).
- We can **check** a candidate quickly.
- We want a candidate that passes the check.

Examples

- Which password opens this file?
- Which assignment of n true/false variables satisfies these rules?
- Which route is shorter than 500 km?

The common idea

Checking is easy, finding is hard.

A checker is short code, but the list of candidates can be astronomically long.

n bits give $N = 2^n$ candidates:
$n = 20$: about a million.
$n = 40$: about a trillion.

Classical search: structure decides the cost

Sorted list: binary search

Compare with the middle, discard half, repeat.

Worst case: about $\log_2 N$ comparisons.
$N = 1023$: at most 10.

Unsorted list: linear search

No order to use: look at the items one by one.

Worst case: N comparisons.
$N = 1023$: up to 1023.

- Grover's algorithm is about the second case: **unstructured search**.
- If your data is sorted, classical binary search is much faster than Grover. Quantum does not beat everything.

The black-box model: we count queries

The only access to the problem is a function

$$f(x) = \begin{cases} 1 & x \text{ is marked} \\ 0 & \text{otherwise} \end{cases}$$

- f is the **oracle**: a black box that recognises a solution.
- One use of f is one **query**.
- Cost = number of queries.

Query cost is not total cost

Each query also costs gates. In practice:

total work = queries$\times$cost per query.

Keep these two apart when you read any speed-up claim.

How expensive is classical search?

Picking cards one at a time, without putting them back, M marked among N:

Expected number of queries

$$\frac{N+1}{M+1}$$

Why: the M marked cards cut the others into $M+1$ gaps of equal average size.

Big-O notation: the classical cost is $\Theta(N/M)$.

Worked example

$N = 31$, $M = 3$: $\frac{32}{4} = 8$ draws.

Check: 28 unmarked in 4 gaps gives 7 before the first marked card; it is draw number 8.

One marked item: $\frac{N+1}{2} \approx \frac{N}{2}$ on average, N in the worst case. $N = 2^{15}$: about 16 384.

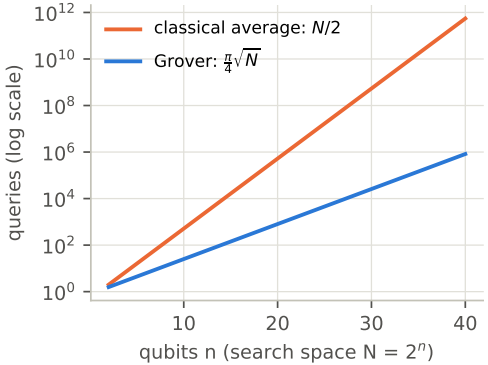
The quantum idea (preview)

Grover's result

Find a marked item with about

$$\frac{\pi}{4}\sqrt{\frac{N}{M}} \quad \text{queries.}$$

N	classical $\frac{N}{2}$	Grover
2^{15}	16 384	142
2^{20}	524 288	804
2^{30}	5.4×10^8	25 735



A **quadratic** speed-up: large, but not exponential.

“Quantum parallelism”: what it does and does not mean

True: one oracle call on a superposition acts on all N basis states:

$$U_\omega \sum_x c_x |x\rangle = \sum_x (-1)^{f(x)} c_x |x\rangle .$$

False: “it tries all answers and gives you the right one”. A measurement returns **one** random x.

The real challenge

After one oracle call each outcome is still equally likely.

We must make the amplitude of the marked item **grow** through interference, a little at each step.

That is why Grover needs $\sim \sqrt{N}$ queries and not just one.

Complex numbers and phases

A complex number $z = a + ib$ with $i^2 = -1$:

- **modulus** $|z| = \sqrt{a^2 + b^2}$ (length)
- **polar form** $z = |z|e^{i\varphi}$, with φ the angle
- **conjugate** $\bar{z} = a - ib$, and $|z|^2 = \bar{z}z$

Euler's formula

$$e^{i\varphi} = \cos\varphi + i\sin\varphi, \qquad |e^{i\varphi}| = 1$$

Worked example

$z = \frac{1+i}{\sqrt{2}} = e^{i\pi/4}$ has modulus 1.

$z^6 = e^{6i\pi/4} = e^{3i\pi/2} = -i$.

The value -1 is a phase: $-1 = e^{i\pi}$. Grover's oracle multiplies amplitudes by exactly this number. It changes the direction, never the length.

Vectors, inner products and normalisation

Quantum states are vectors of complex numbers: a **Hilbert space** is the vector space $\mathbb{C}^N$ together with this inner product:

Inner product

$$\langle u|v\rangle = \sum_k \overline{u_k}\, v_k$$

- Length: $\|v\|^2 = \langle v|v\rangle = \sum_k |v_k|^2$
- Normalised: length 1. Orthogonal: $\langle u|v\rangle = 0$
- Note the bar: conjugate the **first** vector.

Worked example

Normalise $(1, i, 1+i, 2)$:

$|1|^2 + |i|^2 + |1+i|^2 + |2|^2$
$= 1 + 1 + 2 + 4 = 8,$

so multiply by $c = \frac{1}{2\sqrt{2}}$.

Computational basis: the unit vectors $e_0, \ldots, e_{N-1}$. For qubits they are written $|0\rangle, \ldots, |N-1\rangle$. Each is one item of the search space.

The trigonometry we need

Identities

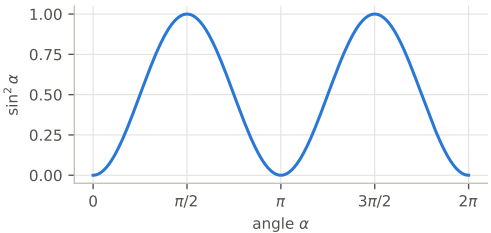
$$\cos 2\alpha = 1 - 2\sin^2\alpha$$

$$\sin 2\alpha = 2\sin\alpha\cos\alpha$$

$$\sin 3\alpha = 3\sin\alpha - 4\sin^3\alpha$$

Worked example

$\sin\theta = \frac{1}{4}$: $\sin 3\theta = \frac{3}{4} - \frac{4}{64} = \frac{11}{16}$, so $\sin^2 3\theta = \frac{121}{256}$.



$\sin^2\alpha$ repeats every π (not 2π) and equals 1 at odd multiples of $\frac{\pi}{2}$. Remember the shape: the success probability of Grover follows it.

Reflections are simple matrices

For a unit vector $|u\rangle$:

Reflection about $|u\rangle$

$$R_u = 2\,|u\rangle\langle u| - I$$

- $|u\rangle\langle u|$ is the matrix with entries $u_j\overline{u_k}$ (the **projector** on $|u\rangle$).
- $R_u\,|u\rangle = |u\rangle$: the axis stays.
- $R_u\,|w\rangle = -\,|w\rangle$ if $\langle u|w\rangle = 0$.
- $R_u^2 = I$: reflecting twice undoes it.

Worked example

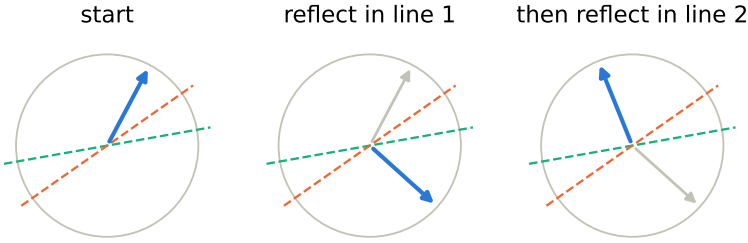
$u = (\frac{3}{5}, \frac{4}{5})$:

$$2uu^T - I = \frac{1}{25}\begin{pmatrix} -7 & 24 \\ 24 & 7 \end{pmatrix}$$

Check: it maps u to u; trace 0, determinant -1.

$|u\rangle\langle u|$ alone is **not** unitary (it has eigenvalue 0); R_u is.

Two reflections make a rotation



lines at 10° (green) and 35° (orange); net effect: rotation by $2 \times 25^\circ = 50^\circ$

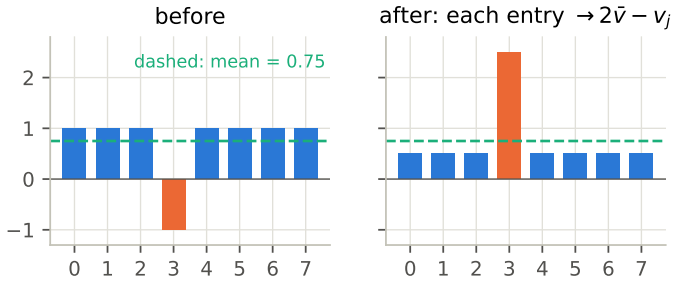
A reflection in the line at angle α sends angle ϕ to $2\alpha - \phi$. Reflect at α, then at β:

$$\phi \to 2\alpha - \phi \to 2\beta - 2\alpha + \phi.$$

Remember this

The result is a rotation by **twice the angle between the lines**. Grover is exactly this.

Inversion about the mean



For a list with mean $\bar{v}$: $v_j \mapsto 2\bar{v} - v_j$. A value a distance d below the mean ends a distance d above it. Example: seven $+1$ and one -1: $\bar{v} = \frac{3}{4}$; the -1 becomes $\frac{5}{2}$, the others $\frac{1}{2}$.

Check

The sum of squares stays the same: $7 \cdot \frac{1}{4} + \frac{25}{4} = 8 = 7 + 1$.

Bit vs. qubit

A classical **bit** is 0 or 1.

A **qubit** is a unit vector of two complex numbers:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

with $|\alpha|^2 + |\beta|^2 = 1$. α, β are called **amplitudes**.

Superposition

$$|\pm\rangle = \tfrac{1}{\sqrt{2}}\big(\,|0\rangle \pm |1\rangle\,\big)$$

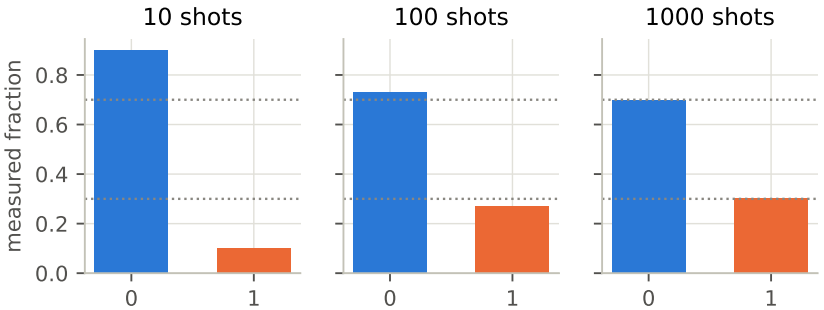
are vectors with two non-zero components — "0 and 1 at the same time" in that precise sense.

The notation $|\cdot\rangle$ ("ket") just marks a column vector; $\langle\cdot|$ ("bra") is its conjugate row vector.

Measurement: amplitudes become probabilities

Born rule

Measuring $\sum_x c_x |x\rangle$ gives x with probability $|c_x|^2$. Afterwards the register *is* in the state $|x\rangle$.



A qubit with $|\alpha|^2 = 0.7$ measured 10, 100 and 1000 times (CUDA-Q). One **shot** gives one bit; the probabilities only appear after many shots (dotted lines: 0.7 and 0.3).

Many qubits: registers and tensor products

- n qubits (a **register**) have 2^n basis states, one for each n-bit string:

$$|\psi\rangle = \sum_{x=0}^{2^n-1} c_x |x\rangle, \qquad \sum_x |c_x|^2 = 1.$$

- Independent qubits combine with the **tensor product**:

$$(a_0|0\rangle + a_1|1\rangle) \otimes (b_0|0\rangle + b_1|1\rangle) = a_0b_0|00\rangle + a_0b_1|01\rangle + a_1b_0|10\rangle + a_1b_1|11\rangle$$

- Example: $\frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ has the amplitude $\frac{i}{2}$ on $|10\rangle$.

Why simulation is hard

A classical computer stores all 2^n complex numbers, 16 bytes each: $n = 30$ needs 16 GiB, and $n = 50$ needs 16 PiB.

Single-qubit gates are 2×2 matrices

NOT gate X

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

$X|0\rangle = |1\rangle$, $X|1\rangle = |0\rangle$

Phase flip Z

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

$Z|0\rangle = |0\rangle$, $Z|1\rangle = -|1\rangle$

Hadamard H

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$H|0\rangle = |+\rangle$, $H|1\rangle = |-\rangle$

- Useful facts: $HXH = Z$, $HZH = X$, $X|\pm\rangle = \pm|\pm\rangle$, $Z|\pm\rangle = |\mp\rangle$.
- Gates are **unitary**: $U^\dagger U = I$. They keep the vector length 1 and can always be undone.
- $U^\dagger$ means "transpose and conjugate". Measurements are not unitary; projectors are not gates.

Hadamard layer: state preparation for search

Apply H to every qubit of $|0\ldots0\rangle$ (the **state preparation** step):

Uniform superposition

$$|s\rangle = H^{\otimes n}|0\rangle^{\otimes n} = |+\rangle^{\otimes n} = \frac{1}{\sqrt{N}}\sum_{x=0}^{N-1}|x\rangle$$

- Every item has amplitude $1/\sqrt{N}$ and probability $1/N$.
- The factor $1/\sqrt{N}$ keeps the total at 1: $N \times \frac{1}{N} = 1$.

General rule:

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{N}}\sum_{y}(-1)^{x\cdot y}|y\rangle\,,$$

with $x \cdot y = \sum_j x_j y_j$. Each input gives a pattern of $\pm$ signs.

The phase: invisible, but it matters

$$|+\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad |-\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Both give 50 % / 50 % when measured — the minus sign is invisible.

But after one more gate:

$$H|+\rangle = |0\rangle, \qquad H|-\rangle = |1\rangle$$

Now the sign decides the result!

Two kinds of phase

Global: $e^{i\gamma}|\psi\rangle$ is the same physical state, always.

Relative: a sign on only *some* amplitudes. Invisible right now, but it matters after interference.

The oracle writes a relative phase. The diffusion turns it into something measurable.

Controlled and multi-controlled gates

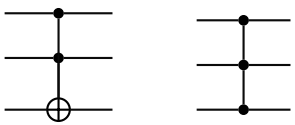
A **controlled-U** applies U to the target only if all controls are $|1\rangle$.

- controlled-X $=$ **CNOT**; doubly controlled X $=$ **Toffoli**
- The **multi-controlled** Z changes one amplitude only:

$$\mathrm{C}^{n-1}\mathrm{Z} = I - 2\,|1\ldots1\rangle\langle1\ldots1|$$

$$= \mathrm{diag}(1,\ldots,1,-1)$$

- It is symmetric: any qubit can be called the target.



Toffoli (left) and CCZ (right): three dots, since all qubits play the same role.

Moving the sign flip with X gates

$\mathrm{C}^{n-1}\mathrm{Z}$ flips only $|1\ldots1\rangle$. To flip another state $|a\rangle$, relabel it:

1. X on each qubit where a has a 0: now $|a\rangle \to |1\ldots1\rangle$.
2. Apply $\mathrm{C}^{n-1}\mathrm{Z}$.
3. Repeat the same X gates to restore the labels.

Rule of conjugation

$$U(I - 2\,|a\rangle\langle a|\,)\,U^\dagger = I - 2\,U\,|a\rangle\langle a|\,U^\dagger$$

A reflection, seen through a unitary, is a reflection about the image.

Worked example

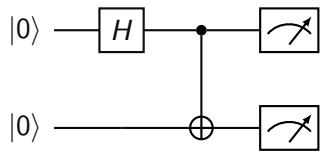
Flip the sign of $|101\rangle$:

apply X_2, CCZ, X_2.

$|101\rangle \xrightarrow{X_2} |111\rangle \xrightarrow{\mathrm{CCZ}} -|111\rangle \xrightarrow{X_2} -|101\rangle$.

Any other state does not reach $|111\rangle$, so it keeps its sign.

Reading a circuit, and entanglement



Product test

$\sum c_{jk} |jk\rangle$ is a product state
$\iff c_{00}c_{11} = c_{01}c_{10}$.

- Each line is a qubit; time runs **left to right**.
- After H: $\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$
- After CNOT: $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$
- Measure: 00 or 11, never 01 or 10. The qubits are **entangled**: $\frac{1}{2} \neq 0$ in the test.
- A sign flip also entangles: $\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle)$ fails the test ($-\frac{1}{4} \neq \frac{1}{4}$). Grover's states are entangled in general.

Grover's algorithm: background and plan

L. K. Grover (1996), *A fast quantum mechanical algorithm for database search*.

The problem

Given black-box access to f, find x with $f(x) = 1$. Assumptions:

- M of the N items are marked, with $M \geq 1$;
- no structure can be used;
- the oracle can act on superpositions.

The plan

1. Start uniform: $|s\rangle$.
2. Oracle: mark the solutions with a sign.
3. Diffusion: turn the sign into a larger amplitude.
4. Repeat 2–3 exactly k times.
5. Measure.

Marked = solutions (M states);
unmarked = the other $N - M$.

The oracle as a circuit: bit oracle and phase kickback

A reversible circuit for f cannot just overwrite; it writes into an extra qubit (an **ancilla**):

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle \qquad (\oplus = \text{XOR}).$$

Phase kickback

Prepare the ancilla in $|-\rangle$:

$$U_f |x\rangle |-\rangle = \frac{1}{\sqrt{2}} |x\rangle \left(|f(x)\rangle - |1 \oplus f(x)\rangle \right)$$

$$= (-1)^{f(x)} |x\rangle |-\rangle$$

If $f(x) = 1$ the bracket swaps its two terms: a minus sign.

Trap

An ancilla in $|+\rangle$ gives *no* sign: $U_f |x\rangle |+\rangle = |x\rangle |+\rangle$.

The ancilla stays unentangled and can be re-used.

The phase oracle

Dropping the unchanged ancilla, the oracle is a sign flip on the solutions:

Phase oracle

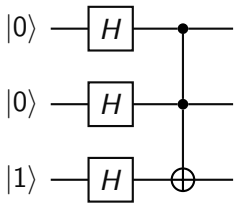
$$U_\omega |x\rangle = (-1)^{f(x)} |x\rangle \qquad \text{and for one marked item:} \qquad U_\omega = I - 2|x^*\rangle\langle x^*|$$

- U_ω is a diagonal matrix with entries ± 1.
- Unmarked: c_x stays. Marked: $c_x \to -c_x$ (**phase inversion**).

The oracle alone reveals nothing

$|-c_x|^2 = |c_x|^2$: all probabilities stay at $1/N$. A measurement now is a random guess.

A real oracle: computing f with kickback



Ancilla in $|-\rangle$; Toffoli computes $f = x_1 \wedge x_2$.

After the gates:

$$\frac{1}{2}\big(\,|00\rangle + |01\rangle + |10\rangle - |11\rangle\,\big)\,|-\rangle$$

- The inputs carry the sign; the ancilla is a separate factor.
- Measuring the ancilla gives 0 or 1 with probability $\frac{1}{2}$ and disturbs nothing.

Counting solutions

$f = (x_1 \vee x_2) \wedge (x_3 \oplus x_4)$: $3 \times 2 = 6$ of 16 inputs, so $\sin^2\theta = \frac{3}{8}$.

The diffusion operator

Diffusion: reflection about $|s\rangle$

$$D = 2\,|s\rangle\langle s| - I$$

Every entry of $|s\rangle\langle s|$ is $\frac{1}{N}$, so

$$D_{xx} = \frac{2}{N} - 1, \qquad D_{xy} = \frac{2}{N}\ (x \neq y).$$

For $N = 4$:

$$D = \frac{1}{2}\begin{pmatrix} -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & -1 \end{pmatrix}$$

Acting on $|\psi\rangle = \sum_x c_x\,|x\rangle$, row x gives:

$$(D\psi)_x = \frac{2}{N}\sum_y c_y - c_x = 2\bar{c} - c_x,$$

with mean $\bar{c} = \frac{1}{N}\sum_y c_y$.

So

D is exactly **inversion about the mean**. It is unitary, because it is a reflection.

One Grover iteration

Grover operator

$$G = D\,U_\omega \qquad \text{(read right to left: the oracle acts first, then the diffusion)}$$

1. **Oracle**: the marked amplitude goes below zero, pulling the mean down a little.
2. **Diffusion**: everything is reflected through the mean. The marked amplitude, far below the mean, lands far above it.

This is amplitude amplification

- Marked: $+a \to -a \to 2\bar{c} + a$
- Unmarked: $+a \to +a \to 2\bar{c} - a$

Each iteration moves amplitude from the unmarked items to the marked one.

Smallest case: two qubits, $N = 4$

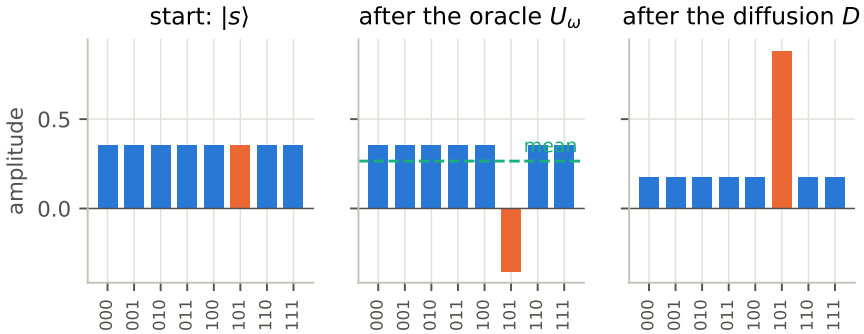
Marked item: any one of the four. All amplitudes start at $\frac{1}{2}$.

step	c_1	c_2	c_3	c_{marked}	mean
start $\lvert s\rangle$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$
after U_ω	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$-\frac{1}{2}$	$\frac{1}{4}$
after D: $2\bar{c} - c$	0	0	0	1	

Result

A single iteration finds the marked item among 4 **with certainty**. A classical search needs 2.25 queries on average (without replacement, stopping when found, and the last one is implied), and there is a real chance of needing 3.

Worked example: three qubits, $N = 8$, $|x^*\rangle = |101\rangle$



With $a = 1/\sqrt{8}$: oracle gives seven $+a$ and one $-a$, with mean $\bar{c} = \frac{6a}{8} = \frac{3}{4}a$. Diffusion: unmarked $2 \cdot \frac{3}{4}a - a = \frac{1}{2}a$; marked $2 \cdot \frac{3}{4}a + a = \frac{5}{2}a$. So $P_1 = \frac{25}{4}a^2 = \frac{25}{32} = 0.781$.

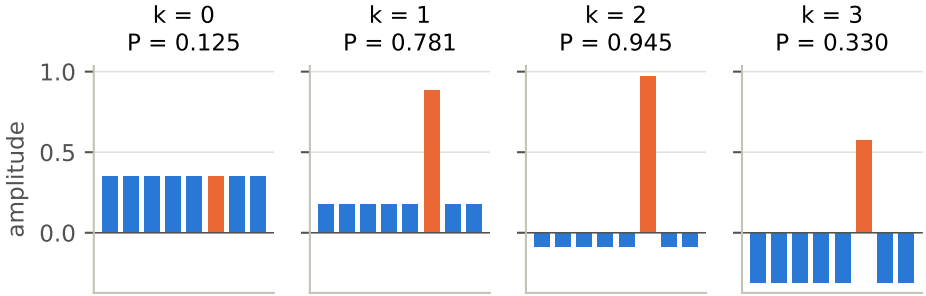
Checking the numbers with CUDA-Q: $N = 8$

after	unmarked c_x	marked c_x	P(marked)
start $\lvert s\rangle$	$+0.3536$	$+0.3536$	0.1250
after U_ω	$+0.3536$	-0.3536	0.1250
after D $(k = 1)$	$+0.1768$	$+0.8839$	0.7812
after U_ω again	$+0.1768$	-0.8839	0.7812
after D $(k = 2)$	-0.0884	$+0.9723$	0.9453

- Simulated with `cudaq.get_state`.
- $0.3536 = a$, $0.8839 = \frac{5}{2}a$, $0.9723 = \frac{11}{4}a$.
- $0.7812 = \frac{25}{32}$, $0.9453 = \frac{121}{128}$.

The second diffusion by hand: the marked value is $-\frac{5}{2}a$, the mean is $\frac{7\cdot\frac{1}{2}a-\frac{5}{2}a}{8} = \frac{a}{8}$, so marked $\to \frac{a}{4} + \frac{5}{2}a = \frac{11}{4}a$ and unmarked $\to \frac{a}{4} - \frac{1}{2}a = -\frac{a}{4}$. The unmarked amplitudes have **gone negative**: the mean is shrinking and the next step will overshoot.

Too many iterations: over-rotation appears



CUDA-Q amplitudes for $N = 8$ (marked item in orange; overall sign removed).

- $k = 2$ is the best choice: $P = 0.945$.
- At $k = 3$ the probability has dropped to 0.330: the search **overshoots**.
- Unlike a classical search, you cannot simply run "a bit longer to be safe".

The algorithm as pseudocode

```
input : oracle for f on n qubits, number M >= 1
output: a marked item, with high probability

theta = arcsin( sqrt(M / 2^n) )
k = round( pi / (4 * theta) - 1/2 )

prepare |0...0>
apply H to every qubit # |s>
repeat k times:
    apply oracle U_omega # flip marked signs
    apply diffusion D # invert about mean
measure all qubits -> x
if f(x) == 1: return x # one classical check
else: run again
```

Why the last check?

Success is probabilistic. A failed run is detected with one more use of f, and you repeat.

The rest of this lecture explains where the formulas for θ and k come from, and how to build the two blocks.

Everything happens in a plane

Define the two normalised states

$$|m\rangle = \frac{1}{\sqrt{M}} \sum_{x \text{ marked}} |x\rangle\,, \qquad |r\rangle = \frac{1}{\sqrt{N-M}} \sum_{x \text{ unmarked}} |x\rangle\,.$$

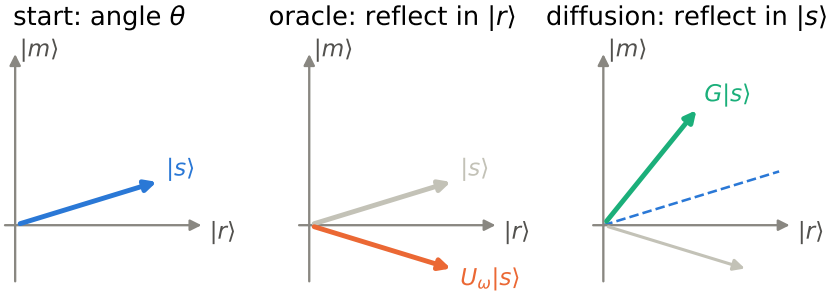
They are orthogonal, and the start state is a combination of them:

Start state in the plane

$$|s\rangle = \sqrt{\frac{N-M}{N}}\,|r\rangle + \sqrt{\frac{M}{N}}\,|m\rangle = \cos\theta\,|r\rangle + \sin\theta\,|m\rangle\,, \qquad \sin\theta = \sqrt{\frac{M}{N}}$$

- θ is the angle between $|s\rangle$ and the unmarked axis. Example: $N = 16$, $M = 1$: $\sin\theta = \frac{1}{4}$, $\theta = 14.5°$.
- Both U_ω and D map this plane to itself, so we can forget the other $N - 2$ dimensions.

The oracle and the diffusion are reflections



- U_ω: fixes $|r\rangle$, negates $|m\rangle$. That is the **reflection in the line through** $|r\rangle$ (angle 0).
- D: fixes $|s\rangle$, negates the perpendicular direction. That is the **reflection in the line through** $|s\rangle$ (angle θ).
- Two reflections with lines θ apart: a **rotation by** 2θ towards $|m\rangle$.

The Grover operator as a matrix

In the basis $\{|r\rangle, |m\rangle\}$:

$$U_\omega = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$

$$|s\rangle = \begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix}, \quad D = \begin{pmatrix} \cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta \end{pmatrix}.$$

Multiplying ($2\cos^2\theta - 1 = \cos 2\theta$, $2\sin\theta\cos\theta = \sin 2\theta$):

Rotation by 2θ

$$G = DU_\omega = \begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix}$$

Eigenvalues

A rotation by 2θ has eigenvalues $e^{\pm 2i\theta}$ and eigenvectors

$$\tfrac{1}{\sqrt{2}}(\,|r\rangle \mp i\,|m\rangle\,).$$

$N = 16$, $M = 1$: $\cos 2\theta = \frac{7}{8}$, $\sin 2\theta = \frac{\sqrt{15}}{8}$; eigenvalues $\frac{7 \pm i\sqrt{15}}{8}$.

Quantum counting will read θ from these eigenvalues.

Deriving the success probability

$|s\rangle$ sits at angle θ, and each G adds 2θ. After k iterations the angle is $\theta + 2k\theta = (2k+1)\theta$:

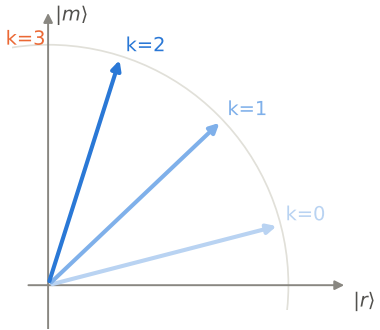
State after k iterations

$$G^k |s\rangle = \sin\left((2k{+}1)\theta\right)|m\rangle + \cos\left((2k{+}1)\theta\right)|r\rangle$$

Measuring gives a marked item with probability $|\text{coefficient of } |m\rangle\,|^2$:

Success probability

$$P_k = \sin^2\left((2k+1)\theta\right)$$



$N = 16$: $\theta = 14.5°$; the angles are $14.5°$, $43.4°$, $72.4°$, $101.3°$. $k = 3$ passes just beyond $|m\rangle$.

Deriving the optimal number of iterations

We want $(2k+1)\theta$ as close as possible to $\frac{\pi}{2}$:

$$(2k+1)\theta = \frac{\pi}{2} \quad \Longrightarrow \quad k = \frac{\pi}{4\theta} - \frac{1}{2}.$$

Optimal number of iterations

$$k_{\text{opt}} = \text{round}\Big(\frac{\pi}{4\theta} - \frac{1}{2}\Big) \approx \frac{\pi}{4}\sqrt{\frac{N}{M}} \qquad (\text{using } \theta \approx \sqrt{M/N} \text{ for } M \ll N)$$

Bound: rounding leaves the angle within θ of $\frac{\pi}{2}$, so

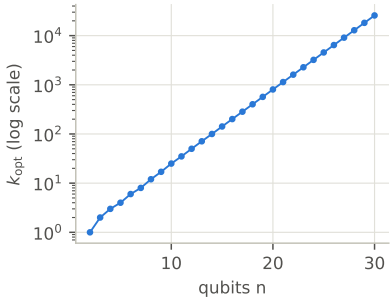
$$P_{k_{\text{opt}}} \geq \cos^2\theta = 1 - \frac{M}{N}.$$

Finite-size trap

$n = 8$: $\frac{\pi}{4}\sqrt{256} = 12.57$ rounds to 13, but the exact rule gives $12.06 \to 12$.
$P_{12} = 0.99995$, $P_{13} = 0.986$.

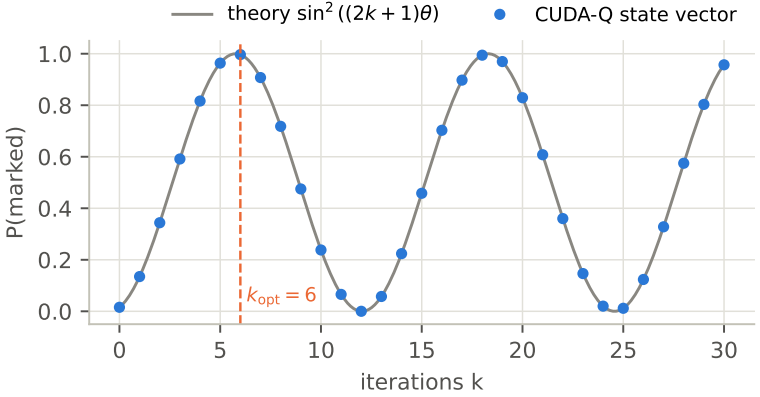
Optimal iterations in numbers

n	N	θ (rad)	k_{opt}	P
2	4	0.5236	1	1.0000
3	8	0.3614	2	0.9453
4	16	0.2527	3	0.9613
5	32	0.1777	4	0.9992
6	64	0.1253	6	0.9966
8	256	0.0625	12	0.9999
10	1 024	0.0313	25	0.9995
15	32 768	0.0055	142	1.0000
20	1 048 576	0.00098	804	1.0000



Optimal does not mean perfect: P is usually slightly below 1. Each extra qubit multiplies k_{opt} by about $\sqrt{2}$, so a straight line on the log scale.

Over-rotation: more is not better



$n = 6$, one marked item (CUDA-Q state vector, largest deviation from theory 6.2e-14). After $k \approx k_{\text{opt}}$ the state rotates **past** $|m\rangle$; at $k \approx 2k_{\text{opt}}$ it is back near $|r\rangle$, and the marked item is **less** likely than a random guess. One full period lasts $\frac{\pi}{2\theta}$ iterations.

Complexity: Grover against classical search

	classical	quantum
queries (one solution)	$\frac{N}{2}$ average	$\frac{\pi}{4}\sqrt{N}$
queries (M solutions)	$\Theta(N/M)$	$\Theta(\sqrt{N/M})$
lower bound	$\Omega(N/M)$	$\Omega(\sqrt{N/M})$
memory	small	n qubits

The ratio of queries is $\frac{N/2}{(\pi/4)\sqrt{N}} = \frac{2}{\pi}\sqrt{N}$. For $n = 20$: 652 times fewer queries.

Optimality

Bennett, Bernstein, Brassard and Vazirani: *any* black-box quantum search needs $\Omega(\sqrt{N})$ queries.
Zalka: $\frac{\pi}{4}\sqrt{N}$ is optimal, including the constant.

Why quadratic and not exponential?

- One iteration rotates by only $2\theta \approx 2/\sqrt{N}$. We must travel about $\frac{\pi}{2}$, so about $\sqrt{N}$ steps are needed.
- The lower bound shows that no cleverness can beat this in the black-box model.
- $2^{n/2}$ is still **exponential in** n. Grover does not make NP-complete problems easy.

Theory versus practice

Theory counts queries.

Practice counts gates and time: each query runs the whole oracle circuit, with error correction on real hardware. The constant factors can absorb much of a quadratic gain.

Several marked items

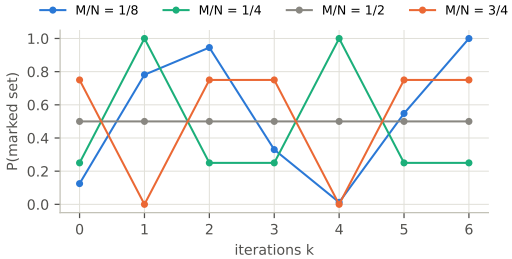
Only the angle changes:

$$\sin\theta = \sqrt{M/N}, \quad k_{\text{opt}} \approx \frac{\pi}{4}\sqrt{N/M}.$$

The final state is the uniform superposition of the solutions, so a measurement returns a random solution.

Worked example

$N = 1024$, $M = 16$: $\sin\theta = \frac{1}{8}$, $\frac{\pi}{4\theta} - \frac{1}{2} = 5.77$, so $k = 6$ and $P = 0.9966$.



Theory curves. $M/N = \frac{1}{4}$: one step is exact. $\frac{1}{2}$: flat at $\frac{1}{2}$. $\frac{3}{4}$: one step erases all marked weight ($P_1 = 0$). If $M > N/2$, just measure. **Note:** the notebook handles one marked item only.

Worked example: two marked items, $N = 8$

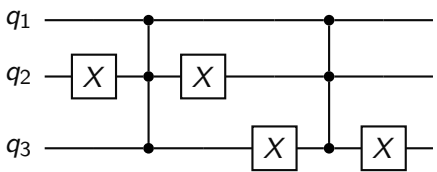
Marked: $|101\rangle$ and $|110\rangle$. Let $a = 1/\sqrt{8}$.

1. Oracle: six $+a$, two $-a$. Mean $\bar{c} = \frac{4a}{8} = \frac{a}{2}$.
2. Diffusion: unmarked $2\bar{c} - a = 0$; marked $2\bar{c} + a = 2a$.
3. Total probability: $2 \times (2a)^2 = 8a^2 = 1$.

Check

$\sin^2\theta = \frac{2}{8} = \frac{1}{4}$, so $\theta = 30^\circ$ and $P_1 = \sin^2 90^\circ = 1$.

Final state $\frac{1}{\sqrt{2}}(|101\rangle + |110\rangle)$: each solution with probability $\frac{1}{2}$.



One oracle per marked string, one after the other; diagonal matrices just multiply their signs.

Theory only: `Grover.ipynb` marks exactly one string.

Unknown number of solutions

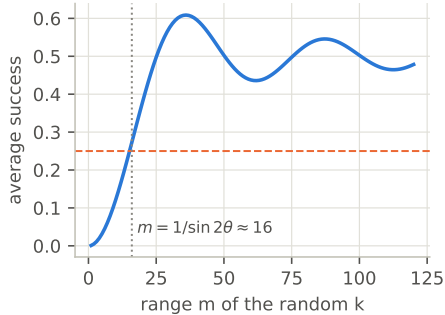
Choosing k needs M. If M is unknown, a fixed k may overshoot. **BBHT** (Boyer, Brassard, Høyer, Tapp) uses a random k:

Average success

$$\frac{1}{m}\sum_{k=0}^{m-1}\sin^2((2k+1)\theta) = \frac{1}{2} - \frac{\sin 4m\theta}{4m\sin 2\theta}$$

This is at least $\frac{1}{4}$ once $m \geq 1/\sin 2\theta$.

Start with $m = 1$; pick k below m; measure and check; if it fails, grow m by a factor such as $\frac{6}{5}$. Expected queries: $O(\sqrt{N/M})$.



$n = 10$, $M = 1$: above $m \approx 16$ the average never falls below $\frac{1}{4}$ (dashed), and it settles at $\frac{1}{2}$.

Exact and fixed-point search

Exact amplitude amplification

$P_{k_{\text{opt}}}$ can miss 1. Fix: round k **up**, then shrink the angle a little so that it lands exactly on $\frac{\pi}{2}$:

$$\theta' = \frac{\pi}{2(2k+1)} \leq \theta.$$

Example $N = 16$: $k = 3$,
$\theta' = \frac{\pi}{14} = 0.224 < 0.253$.

A smaller marked weight (an extra ancilla rotation) gives $P = 1$.

Fixed-point search

Grover's $\pi/3$ method uses reflections with phase $\frac{\pi}{3}$. One recursion turns the failure probability ε into ε^3; extra steps cannot over-rotate.

Example: $0.2 \to 0.008 \to 5.12 \times 10^{-7}$.
The basic form gives up the quadratic speed-up; later variants recover it.

Maximum success is not minimum cost. Repeat-until-success costs k/P_k queries: $N = 16$ gives 2.12, 2.20, 3.12 for $k = 1, 2, 3$, so the cheapest is $k = 1$, not $k_{\text{opt}} = 3$.

Amplitude amplification in general

Grover's trick works for **any** procedure $\mathcal{A}$ (with no measurements) that succeeds with probability a.

Amplitude amplification (Brassard, Høyer, Mosca, Tapp)

$$Q = -\mathcal{A} S_0 \mathcal{A}^{-1} S_\chi, \qquad \sin^2\theta = a, \qquad P_k = \sin^2\left((2k+1)\theta\right)$$

- S_χ flips the sign of good states; $S_0 = I - 2\,|0\rangle\langle 0|$.
- Grover is the case $\mathcal{A} = H^{\otimes n}$, $a = M/N$.
- Uses of $\mathcal{A}$ or $\mathcal{A}^{-1}$: $2k+1$.

Worked example

$a = 0.04$: $\theta = 0.2014$, $k = 3$, 7 uses of $\mathcal{A}$, $P = 0.974$. Classical repetition needs 25 runs on average.

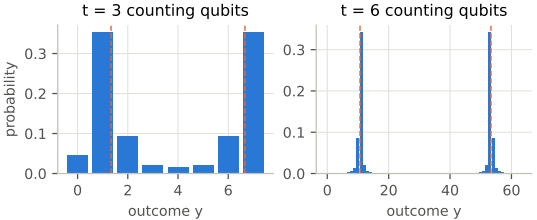
Quantum counting: finding M

Run **phase estimation** on G, starting from $|s\rangle$, an equal mix of both eigenvectors. With $e^{2\pi i\varphi} = e^{\pm 2i\theta}$:

$$\varphi = \frac{\theta}{\pi} \quad \text{or} \quad 1 - \frac{\theta}{\pi}.$$

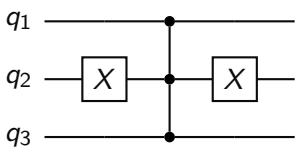
Outcome y on t counting qubits:

$$\tilde{\theta} = \frac{\pi y}{2^t}, \quad \tilde{M} = N \sin^2 \tilde{\theta}.$$



$N = 64$, $M = 16$ (theory, explicit matrices), so $\varphi = \frac{1}{6}, \frac{5}{6}$ (dashed). $t = 3$: the likely $y = 1$ gives $\tilde{M} \approx 9.4$, and $y = 2$ gives 32: too coarse. $t = 6$: peaks at $y = 11$ and 53 give $\tilde{M} \approx 16.9$. More qubits give finer estimates.

The oracle circuit



Oracle for $|x^*\rangle = |101\rangle$

Recipe (no ancillas)

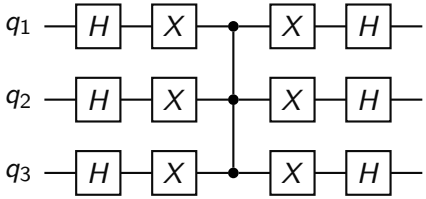
1. X on every qubit where x^* has a 0
2. multi-controlled Z on all n qubits
3. the same X gates again

Result: $U_\omega = I - 2\,|x^*\rangle\langle x^*|$.

Gates: $2z + 1$, with z the number of zeros in x^*.

This is a **demonstration oracle**: it contains the answer. A real oracle would compute f from the problem, and is the expensive part.

The diffusion circuit



Read the middle first, using the conjugation rule:

$$\mathrm{C}^{n-1}\mathrm{Z} = I - 2\,|1\ldots1\rangle\langle 1\ldots1|$$

$$\xrightarrow{X^{\otimes n}} \; I - 2\,|0\ldots0\rangle\langle 0\ldots0|$$

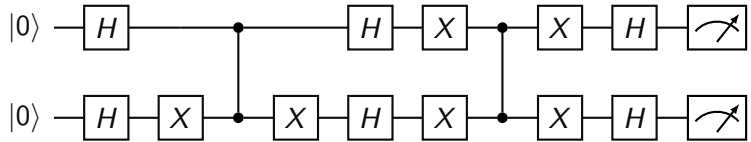
$$\xrightarrow{H^{\otimes n}} \; I - 2\,|s\rangle\langle s| = -D$$

Gates: $4n + 1$.

The minus sign

The circuit implements $-D$, not D. The factor -1 is a **global phase**: it has no observable effect. It is why simulators may show every amplitude with the opposite sign.

The two-qubit Grover circuit, $|x^*\rangle = |10\rangle$



- $n = 2$, so $\theta = 30^\circ$ and $k_{\text{opt}} = 1$.
- With two qubits, the multi-controlled Z is the ordinary CZ.
- Marked string 10 has one zero, on qubit 2.

Gate count

H layer: 2. Oracle: $2 + 1 = 3$. Diffusion: $8 + 1 = 9$. Total 14 gates; formula $n + k(4n + 2z + 2) = 2 + 12$.

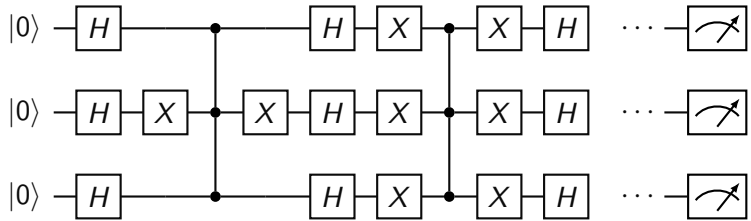
Step-by-step execution of the two-qubit circuit

Amplitudes of $(|00\rangle, |01\rangle, |10\rangle, |11\rangle)$, multiplied by 2 for readability:

block	after gate	$2\times$ amplitudes
start	$\|00\rangle$	$(2, 0, 0, 0)$
preparation	$H \otimes H$	$(1, 1, 1, 1)$
oracle	X on q_2	$(1, 1, 1, 1)$
	CZ	$(1, 1, 1, -1)$
	X on q_2	$(1, 1, -1, 1)$
diffusion	$H \otimes H$	$(1, -1, 1, 1)$
	$X \otimes X$	$(1, 1, -1, 1)$
	CZ	$(1, 1, -1, -1)$
	$X \otimes X$	$(-1, -1, 1, 1)$
	$H \otimes H$	$(0, 0, -2, 0)$

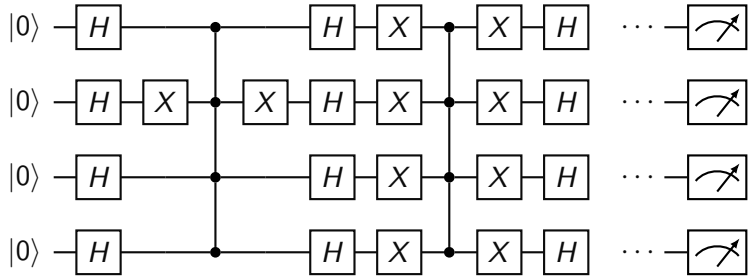
The final state is $-|10\rangle$: measuring returns 10 with probability 1. The minus sign is the global phase from $-D$.

The three-qubit Grover circuit, $|x^*\rangle = |101\rangle$



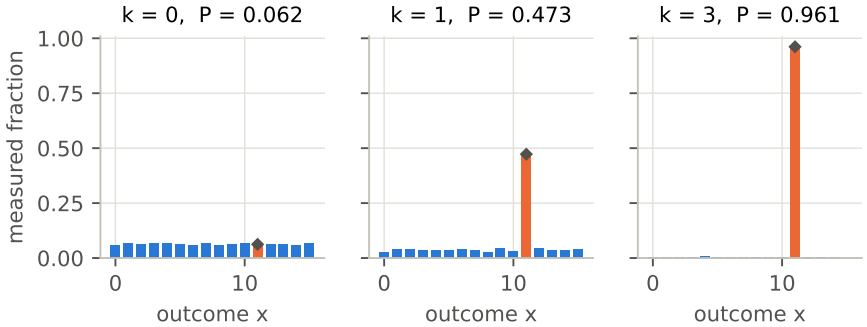
- Oracle: the columns X_2, CCZ, X_2. Diffusion: the next five columns. The dots mean "repeat the two blocks".
- $k_{\text{opt}} = 2$ iterations: $\frac{\pi}{4\theta} - \frac{1}{2} = 1.67 \to 2$. Gates: $3 + 2 \times (12 + 2 + 2) = 35$.
- Probabilities: $P_0 = \frac{1}{8}$, $P_1 = \frac{25}{32}$, $P_2 = \frac{121}{128}$. A third iteration would give only 0.330.

The four-qubit Grover circuit, $|x^*\rangle = |1011\rangle$



- One zero in x^*, so one X pair in the oracle. Now the multi-controlled Z has 3 controls.
- $k_{\text{opt}} = 3$: gates $4 + 3 \times (16 + 2 + 2) = 64$, plus 4 measurements. $P_3 = 0.9613$.

Measurement outcomes: four qubits with CUDA-Q



k	theory P_k	counts	fraction
0	0.0625	123	0.0615
1	0.4727	942	0.4710
3	0.9613	1922	0.9610

2000 shots each; orange marks $|1011\rangle$, the diamond the theory value. Binomial spread: $\sqrt{SP(1-P)}$. At $k = 3$: $\sqrt{2000 \times 0.961 \times 0.039} \approx 9$ counts.

Gate count and circuit depth

With z zeros in x^*, one iteration has:

- $2n$ Hadamards
- $2n + 2z$ X gates
- 2 multi-controlled Z gates

Total

$n + k(4n + 2z + 2)$ gates, plus n measurements. Depth: $2 + 8k$ layers.

Notebook default: $n = 15$, $k = 142$, $z = 6$

- `h`: $15 + 142 \times 30 = 4275$
- `x`: $142 \times 42 = 5964$
- multi-controlled Z: 284
- measurements: 15

Total 10 538 operations and depth 1138: exactly what the notebook reports.

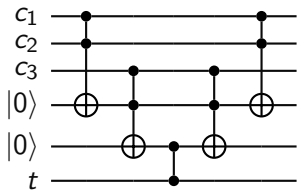
Ancillas and circuit decomposition

A simulator applies $\mathrm{C}^{n-1}\mathrm{Z}$ directly. Hardware cannot: it must be **decomposed** into one- and two-qubit gates.

Textbook construction

A multi-controlled Z with m controls uses $m-1$ clean **ancillas**, $2(m-1)$ Toffoli gates and one CZ.

Example: $n = 10$, $m = 9$: 16 Toffolis per gate, so 32 per iteration.



$m = 3$: compute into two ancillas (4 Toffolis in all), apply one CZ, uncompute.

The repository's notebook uses no ancillas: it calls `z.ctrl` directly.

The notebook: `Grover.ipynb`

What it does

Search for **exactly one** marked bit string, with the oracle built from X gates and `z.ctrl`.

section	content
1	problem and mathematics
3	input parameters (edit only this)
4	validates inputs; θ, k_{opt}
6	expected result, memory, time
7	kernels and circuit drawing
8	execution with memory monitor
9, 10	plots and benchmark report

Default input (Section 3)

```
N_QUBITS = 15
MARKED_BITSTRING = "example"
GROVER_ITERATIONS = "optimal"
RUN_ITERATION_SWEEP = True
SHOTS = 2000
BACKEND = "auto"
SIMULATION_METHOD = "statevector"
```

Notebook source: `build_grover.py` in `dev/notebook_builders/`. Results go to `exports/grover/`.

From formula to code

mathematics	in `build_grover.py`	note
N items	`search_space = 2 ** N_QUBITS`	notebook: M
$\sin\theta = 1/\sqrt{N}$	`theta = math.asin(1.0 / math.sqrt(search_space))`	one marked
k_{opt}	`max(0, round(math.pi / (4 * theta) - 0.5))`	exact rule
$\lvert s\rangle$	`h(qubits)`	all qubits
U_ω	`x, z.ctrl, x`	oracle
$-D$	`h, x, z.ctrl, x, h`	diffusion
P_k	`math.sin((2 * iterations + 1) * theta) ** 2`	theory
$\sqrt{SP(1-P)}$	`math.sqrt(SHOTS * p_theory * (1 - p_theory))`	count spread

Section 4 of the notebook checks the inputs and rejects invalid ones: `N_QUBITS` must be at least 2, and a marked string must have exactly `N_QUBITS` characters 0 or 1.

Oracle and diffusion in code

```
@cudaq.kernel
def grover_iteration(qubits: cudaq.qview,
                     marked: list[int]):
    n = qubits.size()
    # Oracle: multiply the amplitude of |x*> by -1
    for i in range(n):
        if marked[i] == 0:
            x(qubits[i]) # maps |x*> onto |11...1>
    z.ctrl(qubits[0:n - 1], qubits[n - 1])
    for i in range(n):
        if marked[i] == 0:
            x(qubits[i]) # undo the X gates
    # Diffusion D = 2|s><s| - I
    h(qubits) # |s> -> |00...0>
    x(qubits) # |00...0> -> |11...1>
    z.ctrl(qubits[0:n - 1], qubits[n - 1])
    x(qubits) # undo X
    h(qubits) # undo H (result: -D)
```

- `marked` lists the bits of x^*, qubit 0 first.
- `z.ctrl(controls, target)`: the first $n-1$ qubits control a Z on the last: $C^{n-1}Z$.
- `h(qubits)` applies H to the whole register.
- The oracle comes first in the code: $G = DU_\omega$.

The search kernel, shots and results

```
@cudaq.kernel
def grover_search(n: int, marked: list[int],
                  iterations: int):
    qubits = cudaq.qvector(n) # n qubits, all in |0>
    h(qubits) # uniform superposition |s>
    for _ in range(iterations):
        grover_iteration(qubits, marked)
    mz(qubits) # measure in the Z basis

# Execution cell (Section 8), simplified
counts = cudaq.sample(kernels["search"], N_QUBITS,
                      marked_bits, iterations,
                      shots_count=SHOTS)
measured_marked = counts.count(marked_bitstring)
p_measured = measured_marked / SHOTS
most_frequent = counts.most_probable()
```

Expected output

`cudaq.sample` returns bit strings and their counts: for example { 011001010011111:2000 }.

- `count(...)`: shots that returned x^*
- `p_measured`: estimates P_k; the error is about $1/\sqrt{\text{shots}}$ or less
- `most_probable()`: the search result

The iteration sweep: state vector instead of shots

```
stride = max(1, math.ceil(sweep_max / SWEEP_MAX_POINTS))
state = cudaq.get_state(kernels["uniform"], N_QUBITS)
k_done = 0
sweep_iterations.append(0)
sweep_probabilities.append(
    abs(state.amplitude(marked_bits)) ** 2)
while k_done + stride <= sweep_max:
    state = cudaq.get_state(kernels["continue"], state,
                            marked_bits, stride)
    k_done += stride
    sweep_iterations.append(k_done)
    sweep_probabilities.append(
        abs(state.amplitude(marked_bits)) ** 2)
```

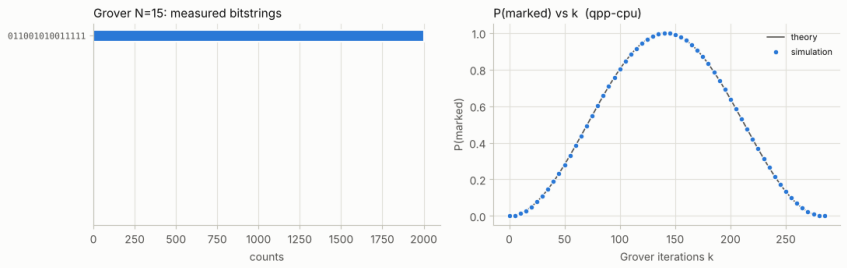
- `get_state`: exact amplitudes, no shot noise.
- The state is handed to the next kernel, so the sweep costs only `sweep_max` iterations.
- $n = 15$: `sweep_max` $= 285$ and `stride` $= \lceil 285/60 \rceil = 5$, so $1 + 57 = 58$ points, as recorded.
- Holding two states doubles the memory: $2 \times 2^{15} \times 16$ bytes $= 1.05$ MB.

Backends: what the notebook offers

BACKEND	SIMULATION_METHOD	CUDA-Q target	status here
cpu	statevector	qpp-cpu	recorded and re-run
cpu	density_matrix	density-matrix-cpu	noise slide only
gpu	statevector	nvidia	**not run**
gpu	mps, tensornet	tensornet-mps, tensornet	**not run**
multi-gpu	statevector	nvidia (mgpu, MPI)	**not run**
hardware	provider	for example quantinuum	**not run**

- A Jupyter kernel is one process, so one GPU. For several GPUs: `jupyter nbconvert -to script Grover.ipynb`, set `BACKEND = "multi-gpu"`, then `mpiexec -np 4 python Grover.py`.
- `BACKEND = "hardware"` with `HARDWARE_EMULATE = True` compiles for a provider and runs locally at no cost.
- The code supports these options; **these slides make no claim about their performance**.

Previously recorded result: 15 qubits



Stored output of `Grover.ipynb` (CUDA-Q 0.16.0, `qpp-cpu`, 4 CPU cores, no GPU). Marked item 011001010011111, $k = 142$: 2000 of 2000 shots correct. Sweep: 58 points, largest deviation from theory 8.2×10^{-13}. Times: execution 23.3 s, sweep 43.3 s, cell total 66.8 s. Peak process memory 0.55 GB.

Expected output, and what it means

```
Grover iterations (k) : 142 (optimal: 142)
Marked bitstring : 011001010011111
Expected result : P(marked) = 0.999987
Actual result : P(marked) = 1.0000 (2000/2000)
Error |P_measured - P_theory| : 0.0000
1-sigma shot noise : 0.0001
Sweep max deviation : 8.20e-13
Search succeeded : YES
Quantum execution time : 23.3304 s
Iteration sweep time : 43.3148 s
Peak process RSS : 0.55 GB (baseline 0.53 GB)
```

Excerpt of the stored benchmark report (Section 10); labels shortened to fit.

- **Expected vs. actual**: theory from $\sin^2((2k+1)\theta)$; measurement from `counts`.
- **Shot noise**: $\sqrt{P(1-P)/S}$ is tiny here because $P \approx 1$.
- **Sweep deviation**: exact amplitudes against theory: only rounding error.
- **Time, memory**: true for that machine, that day. They are not part of the theory.

Newly executed: the same circuit, and how to read it

Re-run for these slides (Linux, 12 logical CPUs, CUDA-Q 0.16.0):

- Gates counted by CUDA-Q: 4275 `h`, 5964 `x`: identical to the notebook.
- Marked counts: 1999 of 2000.
- Run time: 38.6 s (one run).

2000/2000 and 1999/2000 are both consistent with the theory. They are different random samples.

How likely is a perfect run?

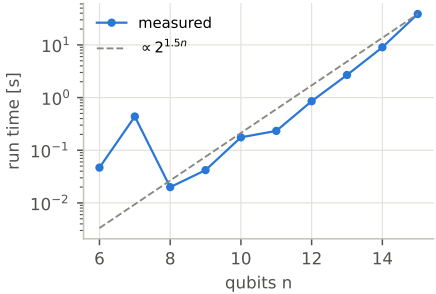
$P = 0.99998683$ per shot, so

$$P^{2000} \approx e^{-2000(1-P)} \approx 0.974.$$

A single failure occurs in about 2.6 % of runs. Even 2000 perfect shots **do not prove** $P = 1$.

Benchmarking: practical performance

n	k	time [s]	marked
6	6	0.05	1990
7	8	0.44	1993
8	12	0.02	2000
9	17	0.04	1997
10	25	0.18	1999
11	35	0.23	2000
12	50	0.85	2000
13	71	2.69	2000
14	100	9.02	2000
15	142	38.58	1999



One run per n on the CPU (2000 shots). Theory: amplitudes double and iterations grow by $\sqrt{2}$ per qubit, so work $\propto 2^{1.5n}$. **Timing depends on the machine**; small n is dominated by overheads, and these are not general performance figures.

Resources as n grows

n	k_{opt}	one state (fp64)	one state (fp32)
15	142	512 KiB	256 KiB
20	804	16 MiB	8 MiB
25	4 549	512 MiB	256 MiB
30	25 735	16 GiB	8 GiB
33	72 792	128 GiB	64 GiB

Computed from $2^n \times 16$ or 8 bytes and the exact rule for k_{opt}. Not measured.

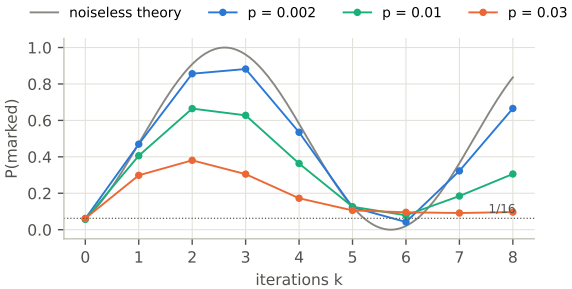
Theoretical vs. practical

Queries grow like $2^{n/2}$.

Simulation work grows like $2^{1.5n}$: a classical simulator touches every amplitude for every gate.

So a simulated Grover run shows the algorithm is correct; it gives **no speed-up over classical search**.

Noise: why long circuits fail



$n = 4$, CUDA-Q `density-matrix-cpu`, 4000 shots per point. Depolarising error p after each H and X gate; multi-controlled gates are left noiseless. Illustrative only: not part of `Grover.ipynb`.

- At $k = 3$: $P = 0.882$, 0.628, 0.305 for $p = 0.002$, 0.01, 0.03 (noiseless: 0.961).
- The signal decays and the state drifts to the uniform value $\frac{1}{16}$.

Simple model

$P_k \approx f^k \sin^2((2k{+}1)\theta) + \frac{1-f^k}{N}$. With $f = 0.99$ and $k = 25$: $P \approx 0.78$.

Hardware considerations and limitations

- **Depth**: $k \propto \sqrt{N}$ iterations, each with large multi-controlled gates. Current devices cannot run this for large n without error correction.
- **Connectivity**: decomposing $\mathrm{C}^{n-1}\mathrm{Z}$ on limited qubit connections adds many two-qubit gates.
- **The oracle**: the demonstration oracle already holds the answer. A useful oracle must compute f reversibly, and can dominate the total cost.
- **Knowing M**: a wrong M causes over-rotation; BBHT or counting are needed if it is unknown.
- **No sorted-data speed-up**: structured problems usually have better classical algorithms.
- **Data loading**: searching a stored database also needs a costly way to query it in superposition.
- **Simulation**: a state vector needs 2^n amplitudes, and the work grows like $2^{1.5n}$.
- **Parallelism**: p machines save only a factor $\sqrt{p}$ in time (classically p).

Applications and future directions

Cryptography

A b-bit key needs about $\frac{\pi}{4}2^{b/2}$ iterations, and each runs the cipher reversibly. Doubling the key length restores the margin.

Optimisation and constraints

Minimum finding (Dürr–Høyer), SAT: a quadratic gain over brute force, not a polynomial algorithm.

Subroutine in other algorithms

Amplitude amplification, and amplitude estimation for Monte-Carlo-style sampling.

task	queries
search, classical	$\Theta(N)$
search, quantum	$\Theta(N^{1/2})$
2-to-1 collision	$\Theta(N^{1/3})$
element distinctness	$\Theta(N^{2/3})$

Directions: resource estimates with error correction, cheaper oracles, fixed-point variants.

Where each result in these slides comes from

kind	what
General theory	Formulas and derivations. Several solutions, unknown M, exact and fixed-point search, amplification, counting, complexity bounds, decomposition with ancillas.
Repository implementation	One marked item; oracle from X and `z.ctrl`; no ancillas; code listings from `build_grover.py`.
Illustrative examples	Hand calculations with small N; the Toffoli oracle; the noise slide (same gates, with a noise model added for this lecture).
Previously recorded	15-qubit run stored in `Grover.ipynb`: 2000/2000, 66.8 s on 4 CPU cores.
Newly executed	CUDA-Q CPU runs by `make_figures.py`: amplitudes, shots, 15-qubit re-run, benchmark, noise.
Not validated	GPU, multi-GPU, MPS, tensor-network and hardware options: present in the code, not run here.

Try it yourself: `Grover.ipynb`

1. Install: `pip install -r requirements.txt`
2. Open `Grover.ipynb` and run all cells.
3. Change only Section 3:
   - `N_QUBITS` (default 15)
   - `MARKED_BITSTRING`: `"example"`, a bit string or an integer
   - `GROVER_ITERATIONS`: `"optimal"` or an integer
   - `BACKEND`: `"cpu"`, `"gpu"`, ...

Experiments

- `N_QUBITS = 4`: $k_{\text{opt}} = 3$ and the whole circuit fits in the drawing
- `GROVER_ITERATIONS = 284`: the marked item almost never appears ($P \approx 3 \times 10^{-6}$)
- `GPU_PRECISION = "fp32"`: half the GPU memory

The notebook compares results with $\sin^2((2k+1)\theta)$ and reports time and memory.

Exercises

1. Trace one iteration by hand for $N = 16$, one marked item. Give the marked amplitude and P_1.
2. Find k_{opt} and P for $n = 5$, one marked item ($\theta = 0.1777$).
3. Which X gates does the oracle for $|0110\rangle$ need? How many gates has one iteration?
4. $N = 64$ with $M = 16$. What is P_1? What about $M = 48$?
5. The diffusion circuit gives $-D$. Why is the result still correct?
6. With $k = 2k_{\text{opt}}$ on 2^{15} items, what do you expect to measure?
7. How much memory does the notebook estimate for $n = 28$ with the sweep on: 4, 8 or 16 GiB?
8. Bonus: show that DU_ω in the $\{|r\rangle, |m\rangle\}$ basis is a rotation by 2θ.

The Grover Question Paper has 80 further questions; the Answer Book explains every option.

Answers

1. Mean after the oracle $\frac{14}{16} \cdot \frac{1}{4} = \frac{7}{32}$. Marked: $\frac{7}{16} + \frac{1}{4} = \frac{11}{16}$. $P_1 = \frac{121}{256} \approx 0.473$.
2. $\frac{\pi}{4\theta} - \frac{1}{2} = 3.92 \to 4$; $P = \sin^2 9\theta = 0.9992$.
3. X on qubits 1 and 4 (the zeros). $4n + 2z + 2 = 16 + 4 + 2 = 22$ gates.
4. $M/N = \frac{1}{4}$: $\theta = 30°$, so $P_1 = 1$. $M/N = \frac{3}{4}$: $\theta = 60°$, so $P_1 = \sin^2 180° = 0$.
5. -1 multiplies the whole state: a global phase, which changes no probability.
6. $569\,\theta = \pi + 0.0017$, so $P \approx 3 \times 10^{-6}$: almost never the marked item.
7. One state $2^{28} \times 16$ bytes $= 4$ GiB; the sweep doubles it: 8 GiB.
8. $U_\omega = \mathrm{diag}(1, -1)$ and $D = \begin{pmatrix} \cos 2\theta & \sin 2\theta \\ \sin 2\theta & -\cos 2\theta \end{pmatrix}$. Multiplying gives $\begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix}$.

Summary

- **Unstructured search**: classically about N/M queries; with Grover about $\frac{\pi}{4}\sqrt{N/M}$.
- **Oracle** U_ω flips the sign of solutions. **Diffusion** $D = 2\,|s\rangle\langle s| - I$ inverts about the mean.
- Two reflections make a **rotation by** 2θ, with $\sin\theta = \sqrt{M/N}$.
- $P_k = \sin^2((2k+1)\theta)$; $k_{\text{opt}} = \text{round}(\frac{\pi}{4\theta} - \frac{1}{2})$; too many iterations **over-rotate**.
- Circuit: H layer, then (X, C^{n-1}Z, X) and (H, X, C^{n-1}Z, X, H), repeated.
- The speed-up is quadratic and provably optimal.

Further reading

- L. K. Grover, *A fast quantum mechanical algorithm for database search* (1996)
- M. A. Nielsen, I. L. Chuang, *Quantum Computation and Quantum Information*, Chapter 6
- Boyer, Brassard, Høyer, Tapp, *Tight bounds on quantum searching* (1998)
- NVIDIA CUDA-Q documentation
- This repository: `Grover.ipynb`, `book/`, `assessments/`